

本小节内容

分析真题代码的时间复杂度

分析真题代码的时间复杂度

2019 年(单链表)

41. (13 分) 设线性表 $L = (a_1, a_2, a_3, \dots, a_{n-2}, a_{n-1}, a_n)$ 采用带头结点的单链表保存, 链表中的结点定义如下:

```
typedef struct node {  
    int data;  
    struct node* next;  
} NODE;
```

请设计一个空间复杂度为 $O(1)$ 且时间上尽可能高效的算法, 重新排列 L 中的各结点, 得到线性表 $L' = (a_1, a_n, a_2, a_{n-1}, a_3, a_{n-2}, \dots)$ 。要求:

- (1) 给出算法的基本设计思想。
- (2) 根据设计思想, 采用 C 或 C++ 语言描述算法, 关键之处给出注释。
- (3) 说明你所设计的算法的时间复杂度。

通过题目我们可以看到真题第 3 问往往都是让计算对应算法的时间复杂度, 那么我们通过上小节的真题代码来分析:

`find_middle` 函数我们可以看到有一个 `while` 循环, 因为 `pcur` 每次移动两个结点, 因此循环的次数是 $n/2$, 忽略首项系数, 所以时间复杂度是 $O(n)$

```
void find_middle(LinkList L, LinkList& L2)  
{  
    L2 = (LinkList)malloc(sizeof(LNode)); // 带头节点的链表  
    LinkList pcur, ppre;  
    pcur = ppre = L->next;  
    while (pcur != NULL)  
    {  
        pcur = pcur->next;  
        if (NULL == pcur)  
        {  
            break;  
        }  
        pcur = pcur->next; // 大哥走两步  
        if (NULL == pcur)  
        {
```

```
        break;
    }
    ppre = ppre->next; // 小弟走一步
}
L2->next = ppre->next; // 让 L2 称为链表后面的一半
ppre->next = NULL; // 让 L 成为前半
}
```

接着我们看下面的 reverse 函数，reverse 函数因为只遍历了 L2 链表，遍历长度是 $n/2$ ，所以时间复杂度是 $O(n)$

```
void reverse(LinkList L2)
{
    LinkList r, s, t;
    r=s=t= L2->next;
    if (NULL == r) // 链表为空
    {
        return;
    }
    s=t= t->next;
    if (NULL == t) // 链表只有 1 个结点
    {
        return;
    }
    t = t->next;
    while (t != NULL)
    {
        s->next = r;
        r = s;
        s = t;
        t = t->next;
    }
    s->next = r;
    L2->next->next = NULL; // 链表的第一个结点的 next 要为 NULL
    L2->next = s; // s 是头部
}
```

接下来我们再看 merge 函数，merge 函数 while 循环遍历次数也是 $n/2$ ，因此时间复杂度是 $O(n)$

```
void merge(LinkList L, LinkList L2)
{
    LinkList pcur, p, q;
    pcur = p = L->next;
    q = L2->next;
    p = p->next;
    while (q!=NULL&& p!=NULL)
    {
        pcur->next = q; // 链表 L 给过来一个结点
```

```
q = q->next; //它就往后走一个
pcur = pcur->next; //新链表往后走一个
pcur->next = p; //链表 L2 给过来一个结点
p = p->next; //它就往后走一个
pcur = pcur->next; //新链表往后走一个
}
if (q != NULL)
{
    pcur->next = q;
}
if (p != NULL)
{
    pcur->next = p;
}
}
```

上面 3 个函数总的运行次数是 $1.5n$ ，忽略首项系数，因此时间复杂度是 $O(n)$